

库存与防超发

业务承诺 / 客诉自愈 / 两桶语义 / Saga 退回 / 启动重建

gomall · 业务 deck 07

今日大纲

库存对五个角色意味着什么

超卖：一笔订单的真实业务代价

两桶库存的业务语义

下单 reserve：业务承诺的工程化

Saga 退回：避免”凭空缺货”事故

关单释放：15 / 30 / 60 分钟的业务权衡

启动重建：服务重启时的业务窗口

客服 SOP：三类高频工单的自查路径

业务承诺 SLO 与超卖应急

业务边界与路线图

库存对五个角色意味着什么

同一个“库存数字”，五个角色五种关心点

角色	关心的事	库存系统该回答的问题
C 端用户	加购了还能不能买	提交订单时不能“突然没货”
商家	我卖了多少 / 还剩多少	哪个 SKU 跌破阈值要补货
运营 / 平台	缺货 SKU 看板 / 补货决策	哪些热品长期 reserved 高
客服	超卖怎么补偿 / 误锁怎么解释	给出可执行话术 + 工单根因定位
SRE	Redis 跟 DB 一致吗	启动重建 / 巡检对账 / 告警分级

”库存”不是一个数

字，是五条不同业务承诺的交叉点。本 deck 顺着这五个角色把每件事摊开。

下单到付款，中间那 15 分钟的业务窗

- 用户点”提交订单”到点”立即支付”，gomall 给一段缓冲时间。
- 这段时间里：商品不能再被别人买走，但也还没真正卖出。
- 业务侧叫”占着但没消耗”，本仓库用 Redis 两桶建模：available / reserved。
- 单桶 stock=N 表达不了这个中间态——要么误锁（用户没付就丢库存），要么超卖（两个人下同一件）。

本 deck 七条业务问题

1. 1 个超卖订单的业务后果到底有多严重？工时、补偿、商誉怎么计？
2. 为什么把库存切成 available + reserved 两桶？业务意义是什么？
3. Redis 重启 / 副本扩容时，库存数字怎么从 DB 重建？期间能下单吗？
4. 下单成功但 DB 落库失败，预占的库存会不会泄漏成“凭空缺货”？
5. 关单（超时 / 用户主动）触发的库存退回，业务上几分钟是合适的？
6. 客服收到“加购后还有 5 件，下单时说缺货”工单，怎么自查 + 怎么回？
7. 库存对外业务承诺（SLO）是什么？哪些场景超过 SLO 就要赔？

超卖：一笔订单的真实业务代价

1 笔超卖订单 = 一连串的钱、工时、品牌损失

环节	业务后果	估算成本
客服工单	用户怒打电话 / 在线投诉, 平均 1 单 25 min	客服工时
用户补偿	退款 + 道歉券 (10 / 30 / 50 元档)	10-50 元
商家发不出货	商家 SLA 违约罚款 / 自掏 腰包补货	商品成本 + 罚金 这是为什么
平台代偿	同档替代品差价 / 自营仓兜 底	差价 + 物流
品牌口碑	社交平台截图传播, 新用户 转化下滑	长期 GMV 损失
单笔 100 元商品超卖	直接现金 $\approx$ 60-150 元 + 不 可量化口碑	远超商品本身利润

库存系统的工程目标是宁可误锁不可超卖——误锁是体验事件（用户去别家买），超卖是法律 / 合同 / 财务 / 品牌四件事一起爆。

零超卖怎么证明：500 个用户同时抢 100 件

业务指标	实测
活动准备库存	100 件
同时下单用户数	500
成功下单数	恰好 100
拒单数（业务码：库存不足）	400
活动结束后 available 桶	0
活动结束后 reserved 桶	100（等待支付）

业务侧解读：“恰好 100”是写死的 CI

红线（少 1 个少卖、多 1 个超卖都不允许）。这条断言代表的不是技术指标，而是对商家“我备货 100 件就只卖 100 件”的承诺。

尾延迟为什么是业务问题：Redis Lua vs DB 行锁

	Redis Lua	DB SELECT FOR UPDATE
平均延迟	1.24 ms	1.29 ms
p95 延迟	3.52 ms	3.65 ms
最差用户体验	136 ms	453 ms
吞吐 (RPS)	51,362	50,142

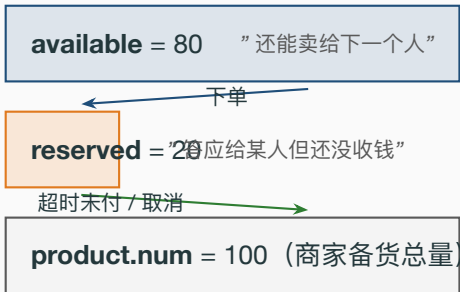
平均 / p95 看不出差距，但最差

那个用户从 136ms 涨到 453ms ——这是 3.3 倍体验差。

业务后果：大促 100k 并发时，DB 锁路径会让尾部几百个用户看到 1-2 秒转圈，转化率掉点；Redis Lua 路径把”最差体验”压在 150ms 以内。

两桶库存的业务语义

available / reserved 翻译成业务话



客服侧的语义价值：当用户问“为什么我加购后看

见还有 5 件，下单却说没货”，客服可以直接答

”其他 5 件被别的客户下单未付款占着，30 分钟内若对方不付款会自动放回来”。

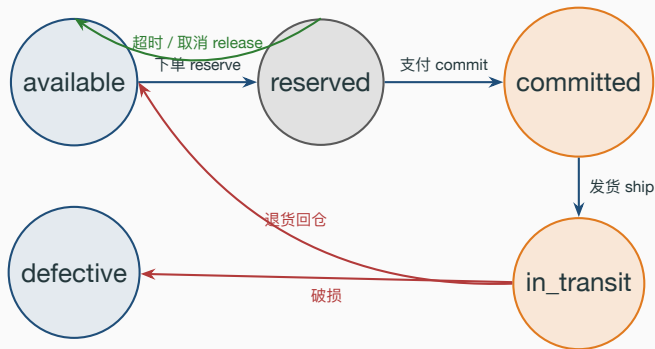
两桶不是技术名词，是对客服可解释的业务模型。

物理库存 vs 可售库存：商家 / 客服 / 财务的语言对齐

桶	谁在看	业务含义
available	C 端 / 商详页	”还能下单的数字”——唯一可对外曝光
reserved	客服 / 商家后台	”已下单未支付”——客服查”为什么缺货”
committed	财务 / 商家	”已支付未发货”——进 GMV / 应发货量
in_transit	物流 (WMS)	在途，归 WMS 仓储系统
defective	仓储 (WMS)	残次，不入可售

业务对齐三原则：(1) C 端只看 **available** ——商详页绝不显示 reserved (会让用户”焦虑性下单”)。(2) 商家后台看 available + reserved ——知道”承诺出去了多少”才好备货。(3) 财务对账盯恒等式： $\text{available} + \text{reserved} + \text{committed} = \text{product.num}$ 。

三态生命周期：业务侧四件事



- 下单 **reserve**：占住一份库存，承诺给该用户。
- 支付 **commit**：钱到账，库存真正消耗，不再可退回 **available**。
- 超时 / 取消 **release**：承诺取消，库存退回再卖给下一个人。
- 发货 **ship** / 退货 / 破损：归 WMS 仓储系统，本仓库不做（路线图）。

下单 reserve: 业务承诺的工程化

8 行 Lua = ”宁可误锁不可超卖”的工程化

```
32 local avail = redis.call('GET', KEYS[1]) -- 1. 读 available
33 if avail == false then
34     return -2 -- 2. 库存未初始化 -> ErrStockNotInit
35 end
36 local need = tonumber(ARGV[1])
37 if tonumber(avail) < need then
38     return -1 -- 3. 不足 -> ErrStockInsufficient
39 end
40 redis.call('DECRBY', KEYS[1], need) -- 4. available -= n
41 redis.call('INCRBY', KEYS[2], need) -- 5. reserved += n
42 return 1 -- 6. 占位成功
```

业务侧关键：第 2 / 第 3 步在扣减之前，宁可拒单也不允许扣穿——这就是”宁可误锁”的工程化。业务码口径：库存不足在代码里是 `cache.ErrStockInsufficient` 错误对象，由上层 handler 统一映射成业务码后下发（注意 50001 是 `ErrorOss` 对象存储错误，与库存无关，两者语义独立）。业务码维度的细分（如把库存类错误单列到 90001 段，与满减引擎占用的 80001-80003 区隔开）属业务码体系的演进项。

下单链路 + 失败自愈：业务侧三步走

链路：用户点提交 → Redis 占库存 → DB 写订单 + outbox → MQ 排关单延迟。

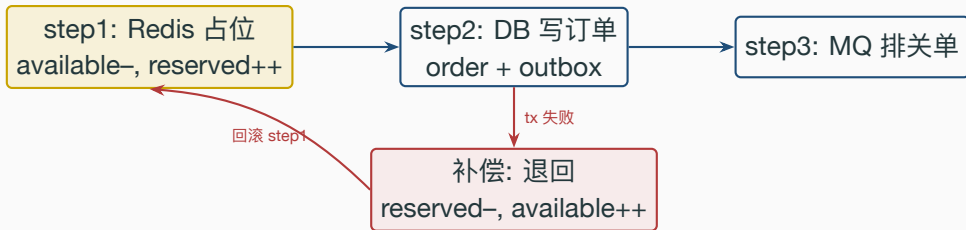
任一步失败：已占的库存必须退回，否则商家会“凭空缺货”。

```
101 // 1) Redis 预扣: available -> reserved, 失败直接拒单
102 if err = cache.ReserveStock(ctx, req.ProductID, int64(req.Num)); err != nil {
103     return nil, err // ErrStockInsufficient
104 }
105 // 2) 同事务写订单 + 应用满减 + 写 outbox (ApplyDiscountInTx 段略)
106 err = dao.NewDBClient(ctx).Transaction(func(tx *gorm.DB) error {
107     if e := NewOrderDaoByDB(tx).CreateOrder(order); e != nil { return e }
108     return outbox.NewOutboxDaoByDB(tx).Insert(/* OrderCreated */)
109 })
110 if err != nil {
111     // 3) 事务失败：必须释放刚扣的预占 (Saga 补偿)
112     if relErr := cache.ReleaseReservation(ctx,
113         req.ProductID, int64(req.Num)); relErr != nil {
114         util.LogrusObj.Errorf("release on tx failure: %v", relErr)
115     }
116     return
117 }
```

先 Redis 后 DB 的业务理由：DB 慢的时候 Redis 直接拒单，不把过载请求压给 DB；DB 是真理之源，热写打进 DB 会让“全平台”一起慢。

Saga 退回：避免“凭空缺货”事故

业务事故：DB 落库失败而预占未退 = 库存泄漏



如果不补偿：available 数字永远比真实可售小，长期堆积所有 **Redis 库存被锁死**——商家明明备了 100 件，前台只显示“还剩 0”——这是真实 P1 事故。

客服话术：用户来投诉时，先查 outbox 是否有同 orderNum 的 OrderCreated 事件 + Redis snapshot 的 reserved 是否偏高，判断是否触发补偿。

commit / release 成对设计的业务理由

```
48 -- commitScript: 支付成功, reserved 真正消耗 (钱到账, 不退)
49 local r = redis.call('GET', KEYS[1]) -- KEYS[1]=reserved
50 if r == false or tonumber(r) < tonumber(ARGV[1]) then
51     return -1
52 end
53 redis.call('DECRBY', KEYS[1], ARGV[1]) -- reserved -= n
54 return 1
55 -- releaseScript: 取消 / 超时, reserved 退回 available
56 local r = redis.call('GET', KEYS[1]) -- KEYS[1]=reserved
57 if r == false or tonumber(r) < tonumber(ARGV[1]) then
58     return -1
59 end
60 redis.call('DECRBY', KEYS[1], ARGV[1]) -- reserved -= n
61 redis.call('INCRBY', KEYS[2], ARGV[1]) -- available += n
62 return 1
```

commit 只动 **reserved** (钱付了, 承诺兑现, 库存不再属于“可退回”池); **release** 两桶都动 (承诺取消, 库存重新可卖)。check $r \geq n$ 避免 reserved 跌负——跌负 = 同一笔订单退了两次 = 库存凭空多出。

**关单释放：15 / 30 / 60 分钟的业务权
衡**

关单超时定多少分钟：商家利用率 vs 用户犹豫时间

档位	用户体验	商家库存利用率
15 分钟	大促时用户来不及付款 / 充值，关单率偏高	库存周转最快，热品多卖一轮
30 分钟（业界中位）	给用户去切支付方式 / 看评论的时间	周转中等
60 分钟	用户犹豫期长，关单率低	热品被锁太久，错过下一波流量

前实现是双保险双口径，各司其职：

- RMQ TTL 延迟队列 OrderCancelDelay = 30 time.Minute：主路径，到点精准投递关单

- Cron 兜底扫表 GetTimeoutOrders(15, 100)：DB 驱动的兜底，扫 WaitPay 超过 15min 的订单

两路都汇到同一个幂等入口 CancelUnpaidOrder：先到者关单，后到者命中 WHERE type=待付款条件 update 自然 no-op，库存不会被退两次。Cron 口径取得更短，是有意让兜底先于主路径触发——即便建单后延迟消息因进程崩溃丢失，DB 扫表仍能在 15min 内补偿关单、退还预占，避免库存被永久占死。

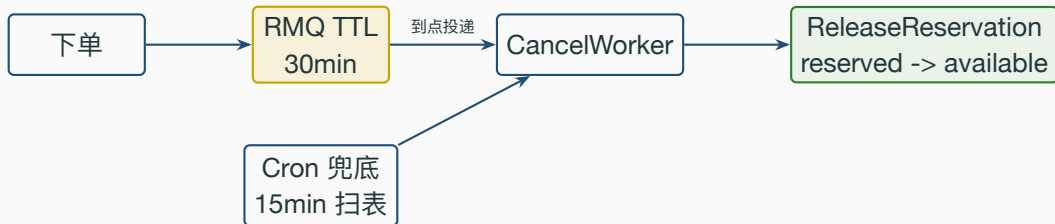
关单释放的幂等保证：客服可以放心”再点一次”

业务背景：客服在工单系统点”立即关单”按钮，可能因为前端超时连点两次；RMQ 重投也会同 orderNum 触发两次；Cron 兜底也会撞上——必须多次执行结果一致，库存不能退第二次。

```
31 err = baseDao.DB.Transaction(func(tx *gorm.DB) error {
32     ok, err := NewOrderDaoByDB(tx).CloseOrderWithCheck(orderNum)
33     if err != nil { return err }
34     if !ok { return nil } // 1. 已关 -> 短路 no-op
35     closed = true
36     return outbox.NewOutboxDaoByDB(tx).Insert(/* OrderCancelled 载荷含 promo 字段 */)
37 })
38 if err != nil { return err }
39 if !closed { return nil } // 2. 没真关 -> 不退库存
40
41 if relErr := cache.ReleaseReservation(ctx,
42     order.ProductID, int64(order.Num)); relErr != nil {
43     util.LogrusObj.Errorf("release on cancel failed: %v", relErr)
44 }
```

closed 标志位是幂等的关键：CloseOrderWithCheck 只对 WaitPay（待付款）生效，二次调用 ok=false 整段 no-op，库存不会退第二次。客服 / RMQ / Cron / 用户主动取消 4 个调用口，统一从这个入口走。

关单延迟链路：MQ 主动 + Cron 兜底

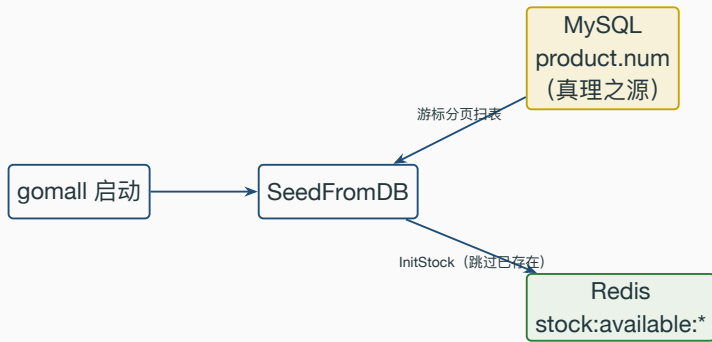


业务侧为什么双保险：

- 单 RMQ：进程重启 / 消息丢 = 漏关 = 商家“凭空缺货”。
- 单 Cron：5min 扫一次，最差用户付完款 5min 才看到 available 回补。
- 双保险：MQ 主路径秒级，Cron 兜底分钟级，两路汇到同一个幂等 `CancelUnpaidOrder` 入口。

启动重建：服务重启时的业务窗口

业务场景：Redis 重启，库存数字从哪儿来？



- 真理之源是 MySQL 的 product.num (商家后台维护，binlog 持久化)。
- Redis 是运行时账本，重启会丢，必须能从 DB 复制回来。
- 已经在跑的实例不能被覆盖 (多副本滚动重启时，新实例不能抹掉旧实例的 reserved)。

SeedFromDB 主循环：游标分页 + EXISTS 跳过

```
14 func SeedFromDB(ctx context.Context, batchSize int) error {
15     if batchSize <= 0 { batchSize = 500 } // 默认页 500
16     d := dao.NewDBClient(ctx); var lastID uint
17     for {
18         var rows []*product.Product
19         q := d.Model(&product.Product{}).Order("id ASC").Limit(batchSize)
20         if lastID > 0 { q = q.Where("id > ?", lastID) } // 游标分页
21         if err := q.Find(&rows).Error; err != nil { return err }
22         if len(rows) == 0 { break }
23         for _, p := range rows {
24             lastID = p.ID
25             exists, _ := cache.RedisClient.Exists(ctx,
26                 cache.StockAvailableKey(p.ID)).Result()
27             if exists == 1 { continue } // 跳过运行期数据
28             cache.InitStock(ctx, p.ID, int64(p.Num)) // 覆盖写 0
29         }
30     }
31     return nil
32 }
```

两个业务保护：(1) 游标分页避免重启时把 DB CPU 打满；(2) EXISTS 跳过保护”运行期已经在卖的实例”，防止新副本起来抹平 reserved。

重建窗口里能不能下单？客服怎么解释？

窗口阶段	C 端体感	客服话术	业务取舍：
0-100s（重建中）	部分商品返” 库存未初始化”	” 系统刚刚重启，1 分钟内会恢复”	
重建完成	全量 available 已回灌	正常下单	
重建失败（DB 慢）	持续返” 库存未初始化”	” 高峰期个别商品暂时不可下单，建议稍后”	

1M SKU 重建 $\approx$ 100s。期间下单返 ErrStockNotInit 触发降级 banner ——宁可少卖 1 分钟也不允许超卖。

SRE 预案：大促前一天手动跑 SeedFromDB 灌全量；活动前 5min 锁商家改库存入口避免双写竞态。

客服 SOP：三类高频工单的自查路径

工单 A: ”我加购后还有 5 件，下单时说缺货”

业务真相：商详页缓存了 5min 前的快照，实际 available 已经被别人下单变成 0。

- **客服自查 3 步：**

1. 查商品 ID → GetStockSnapshot 看 available / reserved;
2. 若 reserved 很高（如 80%）→ 大量预占未付款，30 分钟内会自愈；
3. 若 available = 0 reserved = 0 → 真的卖完了，建议同档替代。

- **话术：**

- reserved 偏高: ”该商品 80% 库存被其他用户下单未付款占着，半小时内若对方不付款会回补，建议您 30 分钟后再试。”

- 真没了: ”该商品已全部售出，为您推荐同款……”

- **自愈机制：**MQ TTL + Cron 双保险确保 30 分钟内一定释放，客服不需要人工介入。

工单 B: ”我下单成功了为什么显示缺货”

业务真相: 该用户的订单成功进了 reserved, 但商详页 available = 0, 用户在订单列表看到”已下单”、在商详页看到”已售罄”, 体感矛盾。

- 客服自查:

1. 查 orderNum → 订单状态 = WaitPay;
2. 查 GetStockSnapshot reserved 包含该用户的 num;
3. 确认是正常占位, 不是事故。

- 话术: ”您的订单已成功提交 (在 *reserved* 状态), 库存已经为您锁定。商详页显示售罄表示其他用户暂时不能再下单。请 30 分钟内完成支付。”
- 业务设计意图: 商详页只看 **available** 是有意的——不能给”焦虑性下单”开口子。

工单 C: ”我退款了为什么库存没还回去”

业务真相: 用户在支付后退款, 资金侧的退款状态机已经完整落地 (internal/refund/service.go, 含 order.refunded outbox 事件按实付口径退款)。库存侧的”从 committed 回到 available”则是有意与退货物流解耦的一段——物理货是否回仓、是否残次, 要等仓储确认, 不能在退款那一刻就盲目把库存放回可售池。

- **现行处理:** 库存回填由商家 / SRE 在确认货品状态后操作 InitStock 覆盖 available (这条路径恪守”误锁可接受、超卖不可”的红线)。
- **话术:** ”您的退款已生效, 库存回填会在 24 小时内由商家确认完成 (避免在途货退回错位)。”
- **扩展方向:**
 - order.refunded outbox 事件已就位 → 由消费侧触发 ReleaseFromCommitted Lua (commit 的逆操作) 做自动回流。
 - 退货寄回 / 残次品分流 → 与 WMS 仓储系统对接。
 - 售后工单 → 评价 → 二次售后的全链路见 deck 09。

业务码 / 错误的客服对照表（含诚实标注）

状态码 / 错误	现行实现	客服话术
ErrStockInsufficient （库存不足）	Go 错误对象，handler 统一映射下发	” 本商品已售罄”
ErrStockNotInit （库存未初始化）	重建窗口内出现	” 系统恢复中请稍候”
60002 ErrIdempotencyInProgress	同 key 重复下单	” 您已下单成功”
70001 ErrRateLimitExceeded	限流命中	” 活动太火爆请稍后”
70002 ErrCircuitOpen	支付熔断 Open	” 支付服务暂忙 1 分钟后”

口径说明：50001 在本仓库是 Error0ss（对象存储错误），与库存语义独立；库存不足由 cache.ErrStockInsufficient 错误对象经 handler 统一映射下发。业务码维度的进一步细分（如把库存类错误单列到 90001 / 90002 段，与满减引擎占用的 80001-80003 区隔），可让客服 / 监控 / 链路追踪更精准地分类。

业务承诺 SLO 与超卖应急

库存子系统的业务 SLO

业务承诺	指标	目标
零超卖	单测 NoOversell 必须绿	100% (CI 红线)
下单库存判断 p99	ReserveStock 端到端	< 10 ms
最差体验	Redis Lua max	< 200 ms (实测)
Saga 退回窗口	DB 失败到 release 完成	< 1s
启动重建 SLA	1M SKU 全量重建	< 120s
关单释放及时性	30min TTL + 15min Cron 兜底	MQ p99 < 31min
对账偏差告警	$\text{available} + \text{reserved} + \text{committed} = \text{product.num}$	偏差 > 1% 告警

”零超卖”在工程上 = `TestInventory_NoOversellUnderConcurrency` 写死 `success ==`

`exactly 100`; 少卖 / 超卖 CI 都红——业务红线工程化。

发生超卖了：4 小时应急 SOP

时间窗	动作	责任方
T+0 min	监控告警：可发货量 < 已支付订单量	SRE on-call
T+5 min	锁定问题 SKU：停下单入口（动态开关）	SRE 改路由
T+15 min	跑对账脚本：超卖量 + 影响订单数	DBA + 库存 owner
T+30 min	客服话术下发：哪些订单发货 / 赔付	客服中心
T+1h	给被超卖用户发 补偿券 + 致歉短信	营销 + 客服
T+4h	复盘 + 修复脚本上线	库存 owner

关键指标：T+5 min 内能锁单是底线。商详页”下单入口”必须是动态开关，不能等发版（动态开关本身也是路线图）。**补偿档位：**< 24h = 退款 + 10 元券；24-72h = 退款 + 30 元券 + 致歉短信；> 72h = 退款 + 50 元券 + 同档替代 + 1v1 客服。

对账恒等式 + 巡检阈值

恒等式: $\text{available} + \text{reserved} + \text{committed} = \text{product.num}$

前两项来自 GetStockSnapshot, $\text{committed} = \text{DB SUM}(\text{num})$ (type 取已付各态, 自

OrderWaitShip 起)。	偏差	处理	校验时
	< 0.1%	日志记录, 不告警	
	0.1%–1%	钉钉提醒, T+1 复查	
	1%–5%	钉钉 P3, 立即排查	
	> 5%	电话 P1, 启动超卖应急 SOP	
	大促前 > 0.1%	拦截活动开始	

机: 10min 抽样 100 个热门 SKU + 凌晨 3:00 全量对账。

修复: SRE 用 InitStock(pid, 真实可售) 人工覆盖; 优先选”误锁”侧而非”超卖”侧——
误锁可挽回, 超卖只能补偿。

业务边界与路线图

本库存系统不做什么（诚实清单）

- **不做多仓库**：当前 Redis key 是 `stock:available:{pid}`，没有 warehouse 维度；多仓分布 + 智能选仓 + 跨仓凑单是路线图。
- **不做 SKU 多规格**：当前以 `product_id` 代 SKU，颜色 / 尺寸 / 套餐没有独立扣减；扩展需新增 `sku_id` + 订单表迁移。
- **不做期货 / 排队购**：普通商品库存必须实物到仓才上架（预售定金 / 尾款两段式已另行落地，见 deck 15）。
- **不做跨仓调拨 / 在途回流**：发货后的在途、退货寄回、残次品分流统一归 WMS 仓储系统，不在本库存模块范围内。
- **不做商家自助补货 API**：商家想加 100 件库存只能找运营手动改 `product.num` + 跑 `InitStock`。
- **不做冻结库存**：质押 / 财务冻结 / 法务冻结这些 B 端需求都没建模。
- **不做防黄牛单用户上限**：库存预扣按件扣减、不绑单用户限购；需要时可直接复用优惠券 `claimScript` 里成熟的 `owned < perUser` 范式 (`repository/cache/coupon.go:30-44`)。

业务路线图：库存能力扩展

能力	业务价值	优先级
独立库存业务码 90001/90002	客服 / 监控 / 链路精准分类	P0
库存预警事件	SKU < 10 件推送商家	P1
商家自助补货 API	商家后台直接加库存	P1
运营缺货 SKU 看板	补货决策可视化	P1
售后库存回流	退款 → committed 退回 avail-able	P1
SKU 多规格扩展	颜色 / 尺寸 / 套餐独立扣减	P2
多仓库 + 智能选仓	就近发货 + 凑单优化	P2
单用户限购 Lua	防黄牛批量下单	P2
秒杀独立 Redis	大促隔离主集群	P2
冷热库存分离	普通商品 / 秒杀物理隔离	P3

级：把库存错误从 Go 错误对象升级到独立业务码——不写代码也能在 5min 内做完。

P1 三件（库存预警 / 自助补货 / 缺货看板）是”商家入驻”业务的核心——直接影响 GMV。

业务承诺：超卖 vs 误锁，两条不同的红线

事故	用户体感	业务代价
超卖	下单成功，无法发货	真金白银赔 + 商誉
误锁	显示“已售罄”	转化率轻微下降

- 超卖 = 法律 / 合同事件，**零容忍**（NoOversell 单测是这条红线的工程化）。
- 误锁 = 体验事件，**可以容忍**（高峰过后 / 关单超时而自愈）。
- 这条不对称是**所有设计选择的源头**：选 Redis 单线程、选先 reserve 后 commit、选 EXISTS 跳过、选先 Redis 后 DB ——全部都在向“宁可误锁不可超卖”这一侧倾斜。

Q&A（上）：业务承诺 / 工单 / 应急

Q1：1 笔超卖订单的真实业务代价多大？

A：100 元商品超卖 $\approx$ 60–150 元直接现金（客服工时 25min + 补偿 10–50 元 + 商家差价 / 罚金 + 平台代偿）+ 不可量化的品牌口碑。这是为什么“零超卖”是 CI 红线。

Q2：用户工单“加购后还有 5 件下单却说缺货”，客服怎么自查？

A：3 步——(1) GetStockSnapshot 看 available / reserved；(2) reserved 高 = 别人占着未付款，30min 自愈；(3) 都为 0 = 真卖完，推同款。

Q3：用户工单“下单成功为什么显示缺货”，是不是事故？

A：不是。该用户已进 reserved，商详只看 available，是有意的产品设计（避免焦虑性下单）。话术：库存已为您锁定，30 分钟内付款。

Q4：发生超卖了应急多久？

A：4 小时 SOP。底线：T+5min 锁定问题 SKU（动态关下单入口），T+30min 发客服话术，T+1h 发补偿券。最关键是“下单入口动态可关”，不能等发版。

Q&A (下): 业务边界 / 路线图 / 诚实标注 i

Q5: 50001 是不是缺货码?

A: **不是**。50001 在本仓库是 `Error0ss` (对象存储错误), 与库存语义独立。库存不足由 `cache.ErrStockInsufficient` 错误对象经 `handler` 统一映射下发; 业务码维度可进一步细分到 90001 段 (与满减引擎占用的 80001-80003 区隔)。代码: `pkg/e/code.go + repository/cache/inventory.go:25`。

Q6: 当前关单超时是 15 还是 30 分钟?

A: **双保险双口径**。RMQ TTL 30min 是主路径精准关单, Cron 兜底 15min 是 DB 驱动的兜底扫表; Cron 口径更短是**有意让兜底先触发**, 两路汇到同一个幂等 `CancelUnpaidOrder`, 后到者 `no-op`、库存不会退两次。代码: `repository/rabbitmq/order_delay.go:22 + internal/order/task.go:24`。

Q7: 用户退款后库存怎么回?

A: 资金侧退款状态机已完整落地; 库存回流**有意与退货物流解耦**, 由商家 / SRE 确认货品状态后用 `InitStock` 覆盖 (恪守“误锁可接受、超卖不可”)。`order.refunded outbox` 事件已就位, 消费侧 `ReleaseFromCommitted Lua` 自动回流是后续扩展项。

Q8: 商家想加库存怎么操作?

Q&A (下): 业务边界 / 路线图 / 诚实标注 ii

A: 当前没有商家自助 API, 得找运营手动改 `product.num` + 跑 `InitStock`。商家自助补货 + 缺货 SKU 看板 + 库存预警事件三件是”商家入驻”P1 路线图。

Q9: 怎么证明 500 并发不会超卖?

A: `NoOversellUnderConcurrency` 单测写死 `success == exactly 100`, 少卖 / 超卖 CI 都红。本质是 Redis 单线程 + Lua 原子。代码: `inventory_test.go:52--82`。

核心实现

reserve / commit / release Lua

InitStock + Snapshot

SeedFromDB + 启动注入

OrderCreate reserve + Saga 补偿

CancelUnpaidOrder release (幂等)

关单口径 (双保险双口径)

RMQ TTL 30 分钟 (主路径)

Cron 兜底 15 分钟 (DB 驱动兜底)

业务码 (口径说明)

50001 = ErrorOss, 与库存语义独立
库存错误统一映射, 可细分 90001 段

测试 & 压测

NoOversell 500 抢 100 = 恰好 100

Lua vs DB 尾延迟 136ms vs 453ms

配套业务 deck: 09-order-lifecycle.tex / 11-consistency.tex / 12-merchant-ops.tex